STAT 578: Mini-Project II

Masked Image Modeling: Masking Design and Target Design

Fall 2026 • Week 8

Overview

In Mini-Project I the pretext task was *invariance*: two views of one image should map to nearby points, and the danger was collapse. Masked modeling takes a different route. There is one view; we hide part of it, ask the model to predict what is missing, and keep whatever internal representation makes that prediction possible. You will again **reproduce**, **probe**, and **interpret** — this time on the two design objects that define a masked model.

Writing $S \sim \pi$ for the set of hidden patches and $\bar{S}$ for the visible ones,

$$\mathcal{L}(\theta) \;=\; \mathbb{E}_{X,\, S\sim\pi}\, \frac{1}{|S|} \sum_{t \in S} \ell\big(q_\theta(X_t \mid X_{\bar{S}}),\, Y_t\big),$$

so a masked method is specified by **the masking distribution** π (how much we hide, and *where*: `--ratio`, `--pattern`) and **the target** Y (what exactly must be predicted: `--target`). MAE and BEiT differ in both; MAE and iBOT differ in Y alone. Everything else — architecture, optimizer, schedule, augmentation — is held fixed by the starter code, which is what turns a flag change into a controlled experiment.

Learning goals. By the end you should be able to (i) write the loss for a given (π, Y) and say what each choice assumes about the data; (ii) explain why the masking ratio is a statement about redundancy rather than a nuisance hyperparameter; (iii) demonstrate that a *lower training loss does not mean a better representation*, and design an evaluation that respects this; (iv) explain why predicting your own features admits a trivial solution, and identify the device that prevents it.

Logistics

- **Groups** of two (self-organized; groups of one are allowed with instructor approval). **Due:** Friday, October 13, 11:59 pm CST, on Canvas.
- **Compute:** CIFAR-10 from scratch on a single modest GPU. **Run everything at `--epochs 100`** — the whole project is **eight runs**, roughly **2–4 hours depending on the GPU**, and fewer epochs while debugging. Keeping one budget for every run is what makes the comparisons fair, and it lets the two runs from Part 1 serve as the reference points for both sweeps in Part 2, so nothing is trained twice.
- **Submit:** a 3–5 page report (PDF), your code, and the required figures. Each group member should contribute to both experimentation and interpretation; the report should briefly describe individual contributions.

- **Starter code** is provided (see below): the masking distributions, a small ViT, both targets, the diagnostics, a training loop, a linear probe, and a reconstruction viewer. You should read and modify it, not treat it as a black box.

The starter code

file	contents
`masking.py`	the masking distribution π: `random`, `block`, `grid`, at any ratio
`vit.py`	small ViT encoder (32×32 image, 4×4 patches, $N = 64$) + light decoder
`methods.py`	`MaskedModel`: targets `pixel` / `feature`; ablation `--no_center`
`data.py`	CIFAR-10 with crop and flip (masked modeling needs little augmentation)
`metrics.py`	`effective_rank`, `output_std`, `cls_patch_sim`, `patch_locality`
`train.py`	pretraining loop; logs `metrics.csv` per epoch; saves `encoder.pt`
`linear_probe.py`	freeze encoder, train a linear classifier, report test accuracy
`visualize.py`	original / masked / reconstruction image grid (pixel target only)

See `README.md` for exact commands. Two structural points are worth reading before you run anything. **The encoder never sees mask tokens** — it runs on visible patches only, and mask tokens enter in the decoder; this is MAE's asymmetry, and it is why a *higher* masking ratio makes training *faster*. **The decoder is thrown away** — only `encoder.pt` is used downstream, which is exactly why reconstruction quality and representation quality can move in opposite directions.

Part 1 — Reproduce

1. Train the two headline configurations at ratio 0.75 with random masking: the **pixel** target (MAE) and the **feature** target (iBOT-style, an EMA teacher). Both are reused as reference points in Part 2 — do not retrain them.

2. For each, run the **linear probe** with both `--pool cls` and `--pool mean`, and report all four numbers. Use `visualize.py` on the pixel model.

3. The reconstructions at 75% masking are blurry. Give a statistical reason why blur is the *expected* output of this objective, not a failure of training.

4. `cls` and `mean` pooling usually differ, often by a lot. Explain the gap in terms of what the objective actually supervises: which tokens receive gradient from the reconstruction loss, and which do not?

Part 2 — Probe (the core of the project)

Design and run experiments; *measure*, do not just train. Record, for every run, the final linear-probe accuracy (using one pooling choice consistently) and the diagnostic trajectories.

(a) The masking distribution π *(five new runs).* Add ratios $\{0.15, 0.50, 0.90\}$ at random masking to the 0.75 run you already have, then, at ratio 0.75, add the two other patterns: `block` (contiguous squares, so some hidden patches have no visible neighbour) and `grid` (checkerboard, so every hidden patch keeps visible neighbours). The pattern comparison removes the same *amount* of information three different ways.

- Plot probe accuracy against ratio, and final pretraining loss against ratio, as separate panels. They do not have the same shape — explain why the loss is *not* comparable across ratios. What changes about the prediction problem itself as the ratio grows?
- At low ratio the task is often solvable by local interpolation. Argue why a locally solvable task yields a weaker global representation, and point to a diagnostic consistent with your argument.
- *Before* looking at your pattern numbers, write down which of `block` and `grid` you expect to win, and why. Then report what happened. A wrong prediction, honestly diagnosed, is worth more here than a lucky right one.

(b) The target Y *(one new run).* The pixel target is a fixed, known function of the input; the feature target is the output of an EMA copy of the encoder, so the right-hand side of the loss moves as training proceeds — which is also why the two losses are numerically incomparable. Compare the two runs from Part 1, then remove the anti-collapse device from the feature target with `--no_center` (DINO-style centering of the teacher).

- Write down the constant solution and verify it minimizes the feature loss. Explain why the pixel loss has no such solution.
- Plot loss, `effective_rank`, `output_std` and `cls_patch_sim` against epoch for the centered and uncentered runs on shared axes. Identify the epoch at which the *loss alone* would have made you declare success, and the epoch at which the *diagnostics* say something is wrong. That gap is the point of the exercise.
- Report the probe accuracy of the uncentered run, and state as precisely as you can what centering does: does it stop the collapsed solution from being a minimizer, or does it make the optimizer less likely to find it? Be careful about what your evidence supports.

Required diagnostics. Every run should report, over training and/or at the end: effective rank, output std, CLS–patch similarity, patch locality, and linear-probe accuracy. Note that effective rank is scale-invariant and so cannot detect a constant representation, whereas output std can; the last two describe the *patch* features, which the linear probe is blind to.

Part 3 — Connect

Interpret your results using lecture material. Address **at least two** of the following:

1. **Redundancy and the ratio.** MAE's 75% is usually justified by "images are redundant." Make that precise enough to be falsifiable: name a quantity, computable from the data and the mask *without training*, that should predict a good ratio, and check it informally against your sweep. Where it fails, say why. If you can, extend it to the pattern: what scalar statistic of π separates `block` from `grid` at equal ratio?
2. **The target as a choice of equivalence.** BEiT predicts discrete tokens, iBOT predicts a teacher's features, MAE predicts pixels. Argue that each is a statement about which images should be treated as the same, place your two targets on that scale, and say what a target that is *too* invariant would look like.
3. **Global versus dense structure.** Existence versus attractiveness returns here: the constant solution survives the feature objective, as it survived BYOL's. Explain how the anti-collapse

mechanism differs from the ones in Mini-Project I, and why `patch_locality` can degrade while probe accuracy holds steady — connecting this to DINOv3's motivation for Gram anchoring.

Deliverables

- **Report (3–5 pages),** workshop-paper style: setup; reproduction results; probing experiments and figures; connection to theory; brief conclusion / limitations.
- **Code** (your modified starter code) with a one-command way to reproduce a key figure, and the `metrics.csv` from every run you report. Do not submit `data/` or checkpoints.
- **Required figures:** (1) probe accuracy and final loss vs. masking ratio, as two panels; (2) the three-pattern comparison at fixed ratio, with reconstruction grids; (3) the collapse panel, centered vs. uncentered. Plus one table of linear-probe accuracy over all eight runs.

Grading rubric

Component	Weight
Reproduction & correctness (both targets train; probe/diagnostics used correctly)	25%
Depth & correctness of the probing analysis (sweeps controlled, diagnostics read right)	35%
Connection to theory (redundancy, target design, existence vs. attractiveness)	25%
Clarity of writing and figures	15%

Weighting the analysis and theory above the reproduction is deliberate: this is a course on *statistical foundations*, not a leaderboard.

Tips & policies

- Start early — the runs are the long pole and they are embarrassingly parallel. Queue them, then write while they finish.
- **Read the diagnostics before you trust the loss.** The most common mistake on this project is to rank configurations by training loss; it is not comparable across ratios, and not comparable at all across targets.
- Fix your seed and say what it is. Differences of a percent or two on CIFAR-10 with a small ViT are within run-to-run noise; if a gap carries your argument, run it twice.
- Defaults in the starter code are a starting point, not tuned optima; absolute accuracy will be modest and the trends across configurations are the object of study.
- **AI policy:** you may use generative AI for debugging and writing clarity, but the experimental design, code, results, and interpretation must be your own understanding, and any AI assistance must be acknowledged (see the syllabus).